

GPT-4o를 이용한 AI 이미지 분석가

영화 <Her>에서는 남자 주인공이 사만다라는 인공지능에게 스마트폰 카메라로 실제 세상을 보여 주며 대화하는 장면이 나옵니다. 오픈AI에서 제공하는 GPT 비전^{vision} 기능을 이용하면 현재 기술로도 이를 구현할 수 있습니다. 이번 장에서는 GPT 비전을 이용하여 이미지에 대한 설명을 작성하고 이를 바탕으로 영어 듣기 평가 문제를 만들어 보겠습니다.

06-1 GPT 비전에게 이미지 설명 요청하기

06-2 이미지를 활용해 퀴즈 만들기

06-1 GPT 비전에게 이미지 설명 요청하기

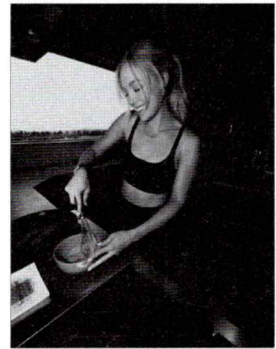
오픈AI 공식 문서를 참고해 GPT 비전 사용법을 익혀 봅시다. 연습할 때는 단계별로 결과를 확인할 수 있는 주피터 노트북이 편리하므로 주피터 노트북 파일로 실습하겠습니다.

◆ GPT 비전 공식 문서: <https://platform.openai.com/docs/guides/vision>

Do it! 실습 인터넷에 있는 이미지 설명 요청하기

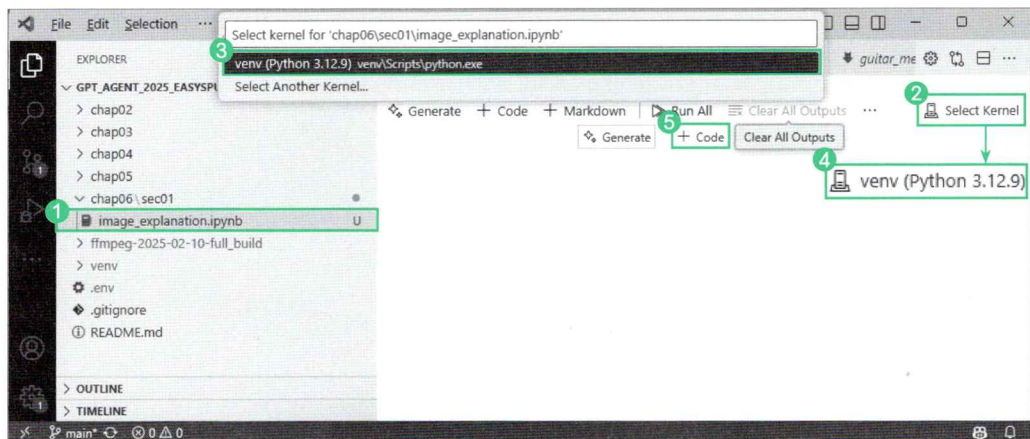
■ 결과 파일: chap06/sec01/image_explanation.ipynb

GPT에게 인터넷에 올라와 있는 이미지를 설명해 달라고 요청해 보겠습니다. 저작권 문제가 없는 언스플래쉬(unsplash.com)에서 이미지 URL를 가져와 사용하겠습니다. 사용할 이미지는 운동복을 입은 외국인 여성이 요리하는 사진입니다. GPT가 이 이미지의 내용을 잘 파악하는지 테스트해 봅시다. 여러분은 인터넷에서 원하는 이미지의 경로를 복사해 사용하면 됩니다.



이미지 출처:
언스플래쉬(Unsplash.com)

1. image_explanation.ipynb 파일을 생성합니다.



2. 첫 번째 셀에 GPT 비전을 활용해 이미지를 분석하는 코드를 작성합니다. GPT API로 챗봇을 만드는 방식과 거의 비슷합니다. 이전에는 `content`에 텍스트를 넣었지만 이 예제에서는 '이 이미지에 대해 설명해 주세요.'라는 텍스트와 함께 `"type": "image_url"`에 사용할 이미지 URL을 넣습니다.



GPT 비전을 이용해 인터넷상의 이미지 설명 받기

image_explanation.ipynb (1)

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기

client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "이 이미지에 대해 설명해 주세요."},
            {
                "type": "image_url",
                "image_url": {
                    "url": "https://images.unsplash.com/photo-1736264335247-8ec-5664c8328?q=80&w=1887&auto=format&fit=crop&ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by-1wYWdlfHx8fGVufDB8fHx8fA%3D%3D",
                    # 사용할 이미지 URL
                }
            },
        ],
    },
]

response = client.chat.completions.create(
    model="gpt-4o", # 응답 생성에 사용할 모델 지정
    messages=messages # 대화 기록을 입력으로 전달
)

response
```

셀을 실행하면 다음과 같이 이미지 내용을 구체적으로 잘 설명해 줍니다.

```
ChatCompletion(id='chatcmpl-AsKmIVhLMsBlJSC20QEbAfkhtRwV', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='이미지에는 한 여성이 주방에서 요리를 준비하는 모습이 담겨 있습니다. 그녀는 검은색 운동복을 입고 있으며 파란색 그릇에 달걀을 휘젓고 있습니다. 근처 프라이팬에는 베이컨이 조리되고 있습니다. 여유롭고 행복한 표정을 짓고 있습니다.'), (... 생략 ...))
```

이렇게 코드를 구성하면 GPT는 인터넷상의 URL에서 이미지를 가져와 분석합니다. 그럼 우리가 찍거나 캡처한 사진을 GPT에게 보내려면 반드시 인터넷에 업로드해야 할까요? 그렇지 않습니다.

Do it! 실습 내가 가진 이미지로 설명 요청하기

■ 결과 파일: sec01/image_explanation.ipynb

오픈AI의 API에 사진 파일을 직접 보낼 수 없으므로 내가 갖고 있는 사진을 설명해 달라고 요청하려면 Base64를 이용해 이미지를 문자열로 변환해야 합니다. Base64는 이진(binary) 데이터를 아스키 문자로 바꾸는 인코딩 방식입니다. 알파벳 대문자(A-Z), 소문자(a-z), 숫자(0-9), 그리고 '+', '/' 같은 특수 문자를 사용해 데이터를 인코딩합니다. 이진 데이터를 Base64로 변환하면 사람이 직접 읽기 편하지는 않지만 텍스트로 표현될 수 있는 형태가 되어 GPT에게 전달할 수 있습니다.

이미지 분석 요청하기

제가 촬영한 망원동에 있는 빵집 사진을 설명해 달라고 GPT에게 요청하겠습니다. 이 사진을 GPT에게 보내기 위해 먼저 Base64로 인코딩을 하겠습니다.

◆ 이 사진은 깃허브에서 내려받을 수 있습니다(https://github.com/saintdragon2/gpt_agent_2025_easyspub/tree/main/chap06/data/images). 물론 여러분이 갖고 있는 사진을 사용해도 무방합니다.



1. `encode_image` 함수는 파일 경로를 매개변수로 받아 이미지 파일을 바이너리 모드(rb)로 열고 base64로 인코딩합니다. 그리고 데이터를 텍스트로 변환한 후 utf-8로 디코딩합니다. 이렇게 해야 JSON 형식으로 이 문자열을 보낼 수 있습니다. 사용할 이미지를 담은 폴더 경로를 `image_path`로 설정하고 이 함수를 실행하여 `base64_image` 변수에 담고 출력합니다.



이미지를 base64로 변환하기

image_explanation.ipynb (2)

```
import base64

# 이미지를 인코딩하는 함수
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

image_path = "../data/images/mangwon_bakery.jpg"

# 이미지를 base64로 인코딩
base64_image = encode_image(image_path)

print(base64_image)
```

셀을 실행하면 다음처럼 문자열로 변환되어 있습니다.

```
/9j/4AAQSkZJRgABAQAAQABAAQ/4QBoRXhpZgAASUkqAAgAAAACAD (... 생략 ...)
```

2. 이제 base64로 인코딩된 이미지를 데이터 URL 형식으로 구성하여 외부 URL 없이 이미지 데이터를 직접 전달할 수 있게 합니다.



base64로 변환한 이미지 설명 요청하기

image_explanation.ipynb (3)

```
messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "이 이미지에 대해 설명해주세요."},
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{base64_image}"
                }
            }
        ]
    }
]
```

```

    },
  },
],
}
]

response = client.chat.completions.create(
    model="gpt-4o",    # 응답 생성에 사용할 모델 지정
    messages=messages  # 대화 기록을 입력으로 전달
)

response.choices[0].message.content

```

이 셀을 실행하면 이미지를 설명하는 내용이 자세히 나옵니다. 저는 이 사진에 직원들이 있는지 몰랐습니다. 이미지를 자세히 보면 쌓여 있는 빵 뒤로 직원들의 모자가 보입니다. 직접 촬영한 저보다 GPT가 낫네요.

'이 이미지는 베이커리 내부로 보이며, 다양한 종류의 빵이 진열되어 있습니다. 진열대 위에는 마카롱 형태의 작은 빵과 크로와상, 롤빵 등 여러 가지 종류의 빵들이 보입니다. 각 빵에는 이름과 가격이 적힌 작은 카드가 함께 놓여 있습니다. 빵은 주로 유리 진열대 안에 놓여 있으며, 고객이 선택할 수 있도록 다양한 종류가 집중적으로 배열되어 있습니다. 뒤쪽으로 직원들이 작업하는 모습도 보입니다. 진열된 빵들은 전체적으로 신선하고 다양해 보이며, 베이커리의 전형적인 내부 모습을 잘 보여주고 있습니다.'

여러 이미지 비교 분석 요청하기

한 번에 여러 이미지를 보내고 그 이미지들을 설명해 달라고 요청할 수도 있습니다. 다음은 테라로사의 선릉점과 홍대서교점을 촬영한 사진입니다. 이 사진들을 활용해 두 카페를 비교해 달라고 요청해 보겠습니다.



테라로사 선릉점



테라로사 홍대서교점

여러 이미지를 설명해 달라고 요청하는 방법도 기존 코드와 크게 다르지 않습니다. 단지 이미
지 여러 개를 딕셔너리 형태로 보내면 됩니다. 각 이미지 경로를 `encode_image` 함수에 인자
로 넣어 `base64`로 인코딩한 후, `messages`에 두 이미지를 딕셔너리 형태로 한 번에 전달합니
다. 그리고 두 카페의 차이점을 설명해 달라는 요청 문구를 추가합니다.



여러 이미지 설명 요청하기

image_explanation.ipynb (4)

```
seolleung_terrarosa_base64 = encode_image("../data/images/seolleung_terrarosa.jpg")
local_stitch_terrarosa_base64 = encode_image("../data/images/local_stitch_terrarosa.
jpg")

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "두 카페의 차이점을 설명해 주세요."},
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{seolleung_terrarosa_base64}",
                },
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{local_stitch_terrarosa_base64}",
                },
            },
        ],
    }
]

response = client.chat.completions.create(
    model="gpt-4o",      # 응답 생성에 사용할 모델 지정
    messages=messages    # 대화 기록을 입력으로 전달
)

response.choices[0].message.content
```

다음은 셀을 실행한 결과입니다. 사람이 직접 작성했다고 해도 믿을 정도로 잘 설명해 주었습니다.

'첫 번째 카페와 두 번째 카페의 차이점을 설명하겠습니다.

1. ****분위기와 인테리어****:

- 첫 번째 카페는 어두운 톤의 조명과 나무 마루 바닥이 사용되어 따뜻하고 아늑한 분위기를 줍니다. 공간이 넓고 개방적이며, 큰 창문을 통해 자연광이 들어옵니다.
- 두 번째 카페는 더 밝고 현대적인 느낌으로, 밝은 색상과 금속 재질의 인테리어가 특징입니다. 바닥은 타일로 되어 있으며, 내부는 넓고 깔끔합니다.

2. ****가구 배치****

- 첫 번째 카페는 원형 테이블과 의자가 규칙적으로 배치되어 있어 그룹 모임에 적합합니다.
- 두 번째 카페는 다양한 크기의 테이블이 있으며, 카운터와 벤치형 좌석이 있어 개인 또는 소규모 그룹의 시간을 보내기에 좋습니다.

3. ****조명****:

- 첫 번째 카페는 조명이 부드럽고 은은하게 설정되어 있어 조용한 분위기입니다.
- 두 번째 카페는 조명이 밝고 균일하게 비치며, 활기찬 환경을 제공합니다.

이와 같은 차이점들이 각 카페의 독특한 느낌을 만들어냅니다.'

★ **한 걸음 더!**

Base64의 장점 3가지

Base64 인코딩은 바이너리 데이터를 텍스트로 안전하게 전송하고 저장하는 방법입니다. Base64 인코딩의 장점을 알아보겠습니다.

- **전송·저장 시 호환성**: 네트워크나 프로토콜에 따라 이진 데이터를 직접 전송하기 어려운 경우가 있습니다. 예를 들면 텍스트 전송만 허용하는 환경에서는 이진 데이터를 전송하기 어렵습니다. 이미지를 포함한 각종 파일을 텍스트 형태로 변환하면 다양한 환경에서 아무 문제없이 데이터를 전송하거나 저장할 수 있습니다.
- **데이터 손상 방지**: 이진 데이터를 그대로 전송하거나 저장하면 문자를 해석하는 인코딩과 디코딩 과정에서 데이터가 손상될 수 있습니다. Base64로 인코딩하면 아스키^{ASCII} 문자로만 이루어져 있어 문자 인코딩 충돌 없이 안전하게 주고받을 수 있습니다.
- **프로토콜과의 호환성**: 이메일 전송이나 특히 MIME에서는 본문에 이미지나 첨부 파일을 삽입할 때 Base64 방식을 자주 사용합니다. RESTful API 같은 웹 API에서도 이미지나 파일을 JSON 형식으로 전송해야 할 때 Base64로 변환하여 전달합니다.

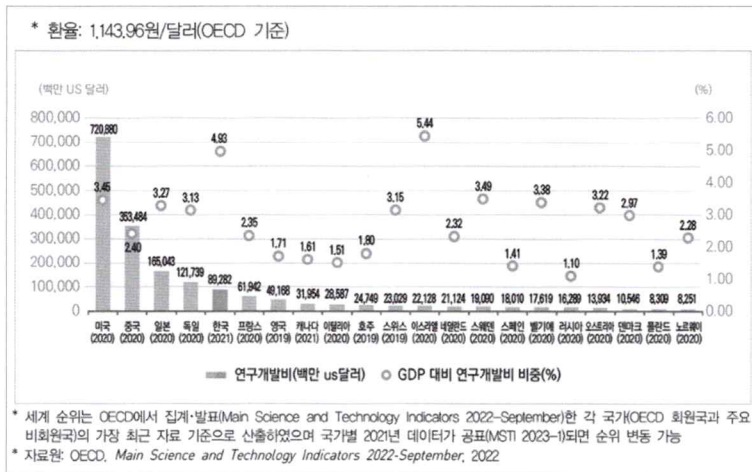
◆ MIME(Multipurpose Internet Mail Extensions)은 이메일에서 이미지, 오디오 등 다양한 형태의 파일을 전송할 수 있게 해주는 표준입니다.

Do it! 실습 GPT 비전의 한계 알아보기

결과 파일: sec01/image_explanation.ipynb

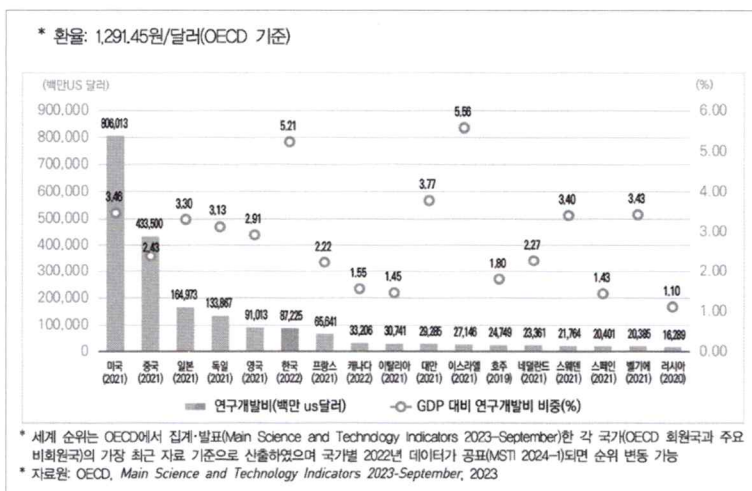
GPT는 이미지를 잘 설명해 주지만 항상 완벽하지는 않습니다. 따라서 정확성을 요구하는 작업에서 GPT를 활용할 때는 주의해야 합니다. 예를 들어 그래프 이미지를 설명해 달라고 요청하면 대부분 제대로 답변하지만 가끔 알 수 없는 이유로 실패하는 경우도 있습니다. 그래프 분석 사례를 통해 GPT 비전의 한계를 알아보겠습니다.

다음은 한국과학기술기획평가원(KISTEP)에서 발간하는 보고서에 수록된 그래프입니다. 이 그래프는 2021년과 2022년 OECD 가입국의 연구개발비를 비교한 내용을 담았습니다.



이 두 그래프 이미지 파일은 이 책의 깃허브에서 내려받을 수 있습니다(https://github.com/saintdragon2/gpt_agent_2025_easyspub/tree/main/chap06/data/images).

연구개발비 국제비교, 연구개발활동조사 2021, KISTEP(파일명: oecd_rnd_2021.png)



연구개발비 국제비교, 연구개발활동조사 2022, KISTEP(파일명: oecd_rnd_2022_large.png)

GPT 비전을 사용해 2021년과 2022년 그래프를 비교 분석해 보겠습니다. 그중 2021년 그래프는 해상도가 다른 이미지 2개를 사용해 각각 어떤 결과가 나오는지 확인해 보겠습니다.

1. 먼저 2021년과 2022년의 연구개발비 그래프를 비교해 달라고 요청합니다. 바로 앞에서 실습할 때 사용한 코드와 거의 동일하며, 이번 실습에서 사용하는 이미지 파일과 요청 사항만 변경했습니다. 여기서는 1047*648 픽셀로 구성된 `oecd_rnd_2021_large.png` 이미지 파일을 사용했습니다.

연구개발비 비교하기 (1) image_explanation.ipynb (5)

```
oecd_rnd_2021_base64 = encode_image("../data/images/oecd_rnd_2021_large.png")
oecd_rnd_2022_base64 = encode_image("../data/images/oecd_rnd_2022.png")

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "첫 번째는 2021년 데이터이고, 두 번째는 2022년 데이터입니다. 이 데이터에 대해 설명해 주세요. 어떤 변화가 있었나요? 한국 중심으로 설명해 주세요."},
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{oecd_rnd_2021_base64}"
                },
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{oecd_rnd_2022_base64}"
                },
            },
        ],
    },
]

response = client.chat.completions.create(
    model="gpt-4o",      # 응답 생성에 사용할 모델 지정
    messages=messages    # 대화 기록을 입력으로 전달
)

response.choices[0].message.content
```

셀을 실행하면 다음과 같이 결과가 출력됩니다. Y축이 양쪽으로 2개나 있는 꽤 복잡한 그래프인데 2021년 그래프는 잘 이해한 것 같습니다. 그러나 2022년 그래프에서는 영국의 수치를 한국의 데이터로 착각했습니다. 한국의 값은 87,225백만달러입니다. 여러분이 실행하는 시점에는 더 잘될 수도 있겠지만 GPT가 복잡한 이미지를 인식하는 데에는 한계가 있으므로 주의해야 합니다.

'두 이미지에서 나타나는 2021년과 2022년의 연구개발비(R&D) 데이터에 대해 설명드리겠습니다. 한국을 중심으로 변화 내용을 살펴보면 다음과 같습니다.

1. ****연구개발비 변화:****

- ****한국의 연구개발비:****

- 2021년 데이터에서는 89,282백만 달러였습니다.
- 2022년 데이터에서는 91,013백만 달러로 증가했습니다.
- 전반적으로 연구개발비에서 약간의 증가가 관찰됩니다.

2. ****GDP 대비 연구개발비 비중 변화:**

- ****한국의 GDP 대비 연구개발비 비중:****

- 2021년에는 4.93%였습니다.
- 2022년에는 5.21%로 증가했습니다.
- 비중 또한 증가하여, 한국의 GDP 대비 연구개발비 비중이 더 올라간 것을 알 수 있습니다.

3. ****국제 비교:****

- 두 해 모두 한국은 연구개발비 기준으로 5위 내외를 유지하고 있으며, GDP 대비 비중에서도 높은 편에 속합니다.

한국은 연구개발비 및 GDP 대비 연구개발비 비중 모두 증가하면서 과학기술 투자에 지속적으로 힘쓰고 있는 모습을 보여줍니다. 이는 한국의 기술 발전과 경제 성장 동력으로 작용할 가능성을 시사합니다.'

2. 이번엔 `oecd_rnd_2021_large.png` 대신 해상도가 조금 낮은 `oecd_rnd_2021_medium.png` 파일을 사용해 봅시다. 이 이미지는 895*538 픽셀입니다. 이전 코드에서 `oecd_rnd_2021_large.png`를 `oecd_rnd_2021_medium.png`로 바꿔서 실행합니다.

 연구개발비 비교하기 (2)

 `image_explanation.ipynb` (6)

```
oecd_rnd_2021_base64 = encode_image("../data/images/oecd_rnd_2021_medium.png")
oecd_rnd_2022_base64 = encode_image("../data/images/oecd_rnd_2022.png")
(... 생략 ...)
```


GPT-4o 모델은 조금 실망스러운 결과를 보여 줍니다. GPT는 2021년의 연구개발비를 121,739백만 달러라고 잘못 읽었습니다. 이 값은 그래프에서 한국 왼쪽에 위치한 독일의 연구개발비에 해당합니다. 이 숫자를 잘못 읽으면서 GDP가 상승했을 것이고 연구개발비 전략 조정이 있었을 것 같다는 잘못된 추측을 하고 있습니다.

'이 두 데이터는 각각 2021년과 2022년의 OECD 주요 국가들의 연구개발비 규모와 GDP 대비 연구개발비 비중을 비교한 자료입니다. 한국을 중심으로 보면 다음과 같은 변화가 있었습니다:

1. **연구개발비 (백만 US 달러):**

- 2021년: 121,739

- 2022년: 87,225

연구개발비가 감소한 것으로 보입니다.

2. **GDP 대비 연구개발비 비중 (%):**

- 2021년: 4.93%

- 2022년: 4.93

GDP 대비 연구개발비 비중은 그대로 유지되었습니다.

대한민국은 연구개발비 총액 면에서는 감소했지만, GDP 대비 비중은 변동이 없는 것으로 나타났습니다. 이는 GDP 상승 또는 연구개발비 전략의 조정에 따른 결과일 수 있습니다.

또한, 다른 주요 국가들과 비교했을 때 미국과 중국이 가장 높은 연구개발비를 유지하고 있으며, 한국은 두 나라에 비해 낮은 수치를 기록하고 있습니다. 연구개발비 비중 측면에서는 이스라엘이 가장 높은 비중을 나타냈습니다.

전체적으로 이러한 변화는 각국의 경제 상황과 연구개발에 대한 정책 변화에 영향을 받을 수 있음을 시사합니다.'

이처럼 GPT의 이미지 인식 기능은 일반적인 이미지 분석에는 적절할 수 있지만 그래프를 해석하거나 환자 CT 사진에서 질병을 감지하는 등의 고차원적인 목적에는 적합하지 않을 수 있습니다. 심지어 완전히 동일한 이미지인데도 이미지 크기에 따라 완전히 잘못된 답변을 내놓을 수 있으므로 반드시 주의해야 합니다. 여러분이 이 책을 읽는 시점에는 GPT가 개선되어서 여기에서 보여 준 실수를 더 이상 하지 않을 수도 있습니다. 그러나 GPT의 이미지 인식 기능에 한계가 있다는 점은 인지해 두기 바랍니다.

06-2 이미지를 활용해 퀴즈 만들기

이미지를 분석해 영어 듣기 평가 문제를 만들어 봅시다. 문제를 영어로 만들면 결과를 확인하기 어려우니 우선 한국어로 만들어 보겠습니다.

Do it! 실습 문제 생성 함수 만들기

■ 결과 파일: sec02/image_quiz_0.py

1. 새로운 파이썬 파일을 만들고 06-1절에서 사용한 image_explanation.ipynb 코드를 활용하겠습니다. 앞으로 for 문으로 여러 이미지 파일의 경로를 가져오기 위해 파이썬 라이브러리 glob를 미리 임포트합니다.

이미지 경로를 받아 base64로 인코딩하는 함수

image_quiz.py

```
from glob import glob # for 문으로 여러 파일의 경로를 가져오기 위해 선언
from openai import OpenAI
from dotenv import load_dotenv
import os
import base64

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기
client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")
```

2. 문제를 출제하는 image_quiz 함수를 만듭니다.

이미지 경로를 받아 퀴즈를 만드는 함수

image_quiz.py

```
(... 생략 ...)
def image_quiz(image_path):
    base64_image = encode_image(image_path) ❶
```

```

quiz_prompt = """
제공한 이미지를 바탕으로, 다음과 같은 양식으로 퀴즈를 만들어 주세요.
정답은 (1)~(4) 중 하나만 해당하도록 출제하세요.
아래는 예시입니다.
----- 예시 -----

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?
- (1) 베이커리에서 사람들이 빵을 사는 모습이 담겨 있습니다.
- (2) 맨 앞에 서 있는 사람은 빨간색 셔츠를 입었습니다.
- (3) 기차를 타기 위해 줄을 서 있는 사람들이 있습니다.
- (4) 점원은 노란색 티셔츠를 입었습니다.

정답: (4) 점원은 노란색 티셔츠가 아닌 파란색 티셔츠를 입었습니다.
(주의: 정답은 (1)~(4) 중 하나만 선택하도록 출제하세요.)
=====
"""

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": quiz_prompt},
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{base64_image}",
                },
            },
        ],
    },
]

response = client.chat.completions.create(
    model="gpt-4o", # 응답 생성에 사용할 모델을 지정
    messages=messages # 대화 기록을 입력으로 전달
)

return response.choices[0].message.content

```

- ❶ 이 함수는 이미지의 경로(image_path)를 매개변수로 받아 encode_image 함수를 이용해 이미지를 base64로 인코딩합니다.
- ❷ 문제를 어떻게 출제해야 할지 예시를 포함해 프롬프트를 작성합니다.
- ❸ messages 구조는 앞서 06-1절에서 사용한 방식과 동일하지만 content의 text 부분에 quiz_prompt가 포함된 점이 다릅니다. 인코딩된 이미지를 GPT에 보낼 때 사용할 프롬프트를 quiz_prompt 변수에 담습니다.
- ❹ 이 함수는 GPT가 생성한 답변에서 content의 텍스트만 추출하여 반환합니다.

3. 함수가 잘 작동하는지 확인해 봅시다. 원하는 이미지를 추가해 테스트합니다.

◆ 예제에서 사용한 busan_dive.jpg는 저자 깃허브에서 내려받을 수 있습니다.

이미지 경로를 받아 퀴즈를 만드는 함수

image_quiz.py

```
(... 생략 ...)
q = image_quiz("./chap06/data/images/busan_dive.jpg")
print(q)
```

실행하니 다음과 같이 문제가 제대로 출력되었습니다. 이제 문제를 만들 이미지 파일들을 원하는 폴더(./data/images/)에 저장하고 for 문으로 반복해서 실행하면 됩니다.

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?

- (1) 많은 사람들이 한 공간에 앉아 컴퓨터 작업을 하고 있습니다.
- (2) 행사명으로 "DIVE 2024 IN BUSAN"이 보입니다.
- (3) 사람들이 서서 기차를 기다리는 모습입니다.
- (4) 천장에는 조명이 많이 설치되어 있습니다.

정답: (3) 사람들이 서서 기차를 기다리는 모습이 아니라, 앉아서 작업 중인 모습입니다.

4. 여러 이미지와 관련된 문제를 생성하고 그 결과를 문제집으로 만드는 코드를 작성합니다.

여러 이미지로 문제집 만들기

image_quiz.py

```
(... 생략 ...)

txt = ''
no = 1
for g in glob('./chap06/data/images/*.jpg'):
    try:
        q = image_quiz(g)
    except Exception as e:
        print(e)
        continue

    divider = f'## 문제 {no}\n\n'
    print(divider)
```

```

txt += divider
filename = os.path.basename(g)
txt += f'![image]({filename})\n\n'

# 문제 추가
print(q)
txt += q + '\n\n-----\n\n'

with open('./chap06/data/images/image_quiz.md', 'w', encoding='utf-8') as f:
    f.write(txt)

no += 1 # 문제 번호 증가

```

- ❶ for 문을 사용해 계속 생성되는 문제들을 덧붙여 나가기 위해 빈 문자열 txt를 선언하고 문제 번호를 매기기 위해 no 변수를 선언합니다.
- ❷ for 문으로 ./data/images/ 폴더 내의 모든 JPG 파일을 사용합니다. 첫 번째로 수행할 작업은 방금 만든 image_quiz 함수를 사용하여 퀴즈를 생성하는 일입니다. GPT가 실수를 하거나 오류를 일으킬 수 있으므로 try 문으로 처리하고, 오류가 발생하면 해당 문제는 출제하지 않도록 단순하게 구현합니다.
- ❸ 마크다운 형식으로 결과를 보기 좋게 출력하는 코드입니다. GPT가 출제한 문제 위에 이미지를 표시하기 위해 이미지 파일명만 추출하여 ![image](filename) 형식으로 링크를 만들고 이를 txt에 덧붙입니다. 문제들 사이에 구분선을 추가해 가독성을 높였습니다.
- ❹ 출제된 문제는 이미지가 있는 폴더(./data/images)에 마크다운 파일로 저장합니다. 이미지 파일이 이미 있는 폴더에 마크다운 파일을 저장하므로 이미지 파일 링크를 만들 때는 전체 경로를 쓸 필요 없이 이미지 파일명만 남겨도 문제없이 연결됩니다. for 문이 종료된 후 문제들을 모두 txt에 담아 마크다운 파일로 저장할 수도 있지만, GPT가 문제를 작성하는 중에도 .md 파일을 실시간으로 열어서 확인할 수 있도록 for 문 안에 파일 저장 코드를 추가합니다.

코드를 실행하면 마크다운 파일이 잘 생성됩니다.

마크다운으로 문제집이 생성된 결과

./chap06/data/images/image_quiz.md

문제 1

![image](busan_dive.jpg)

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?

- (1) 여러 사람들이 큰 실내 공간에서 컴퓨터를 사용하고 있습니다.
- (2) 이미지 상단의 전광판에는 "DIVE 2024 IN BUSAN"이라고 쓰여 있습니다.
- (3) 사람들이 서로 멀리 떨어져 여러 곳에 앉아 있습니다.
- (4) 이미지의 천장 구조물이 노출되어 보입니다.

정답: (3) 사람들이 서로 멀리 떨어져 있지 않고 가까이 모여 앉아 있습니다.

문제 2

![[image](local_stitch.jpg)]

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?

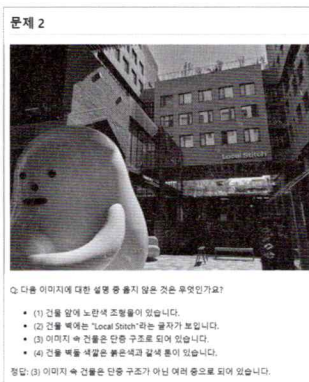
- (1) 건물 앞에 노란색 조형물이 있습니다.
- (2) 건물 벽에는 "Local Stitch"라는 글자가 보입니다.
- (3) 이미지 속 건물은 단층 구조로 되어 있습니다.
- (4) 건물 벽돌 색깔은 붉은색과 갈색 톤이 있습니다.

정답: (3) 이미지 속 건물은 단층 구조가 아닌 여러 층으로 되어 있습니다.

5. VS Code에서 마크다운 파일을 연 뒤 화면 위에서 돋보기 아이콘 을 클릭하면 마크다운 형태로 렌더링된 페이지가 나타납니다.



마크다운 파일을 렌더링한 결과도 마음에 듭니다! 이미지와 함께 그와 관련된 문제도 잘 표시됩니다.



이제 영어로도 문제를 출제해 보겠습니다. 기존 image_quiz.py 파일에 이어서 작성합니다.

1. image_quiz 함수가 영어 버전으로 문제를 출력하고 실패하면 재시도하도록 수정합니다.

영어 버전으로 문제를 출제하고 실패 시 재시도하도록 수정하기

image_quiz.py

```
(... 생략 ...)
```

```
def image_quiz(image_path, n_trial=0, max_trial=3): ①
    if n_trial >= max_trial: # 최대 시도 회수에 도달하면 포기
        raise Exception("Failed to generate a quiz.")

    base64_image = encode_image(image_path) # 이미지를 base64로 인코딩

    quiz_prompt = """
    제공한 이미지를 바탕으로, 다음과 같은 양식으로 퀴즈를 만들어 주세요.
    정답은 (1)~(4) 중 하나만 해당하도록 출제하세요.
    토익 리스닝 문제 스타일로 문제를 만들어 주세요.
    아래는 예시입니다.
    ----- 예시 -----

    Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?
    - (1) 베이커리에서 사람들이 빵을 사는 모습이 담겨 있습니다.
    - (2) 맨 앞에 서 있는 사람은 빨간색 셔츠를 입었습니다.
    - (3) 기차를 타기 위해 줄을 서 있는 사람들이 있습니다.
    - (4) 점원은 노란색 티셔츠를 입었습니다.

    Listening: Which of the following descriptions of the image is incorrect?
    - (1) It shows people buying bread at a bakery.
    - (2) The person standing at the front is wearing a red shirt.
    - (3) There are people lining up to take a train.
    - (4) The clerk is wearing a yellow T-shirt.

    정답: (4) 점원은 노란색 티셔츠가 아닌 파란색 티셔츠를 입었습니다.
    (주의: 정답은 (1)~(4) 중 하나만 선택하도록 출제하세요.)

    =====
    """""

    messages = [
        (... 생략 ...)
    ]
```

```

try:
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        messages=messages # 대화 기록을 입력으로 전달
    )
except Exception as e:
    print("failed\n" + e)
    return image_quiz(image_path, n_trial+1)

content = response.choices[0].message.content

if "Listening:" in content: )
    return content, True
else:
    return image_quiz(image_path, n_trial+1) )
(... 생략 ...)

```

- ❶ 오픈AI의 API 서버가 불안정하거나 부적절한 이미지, 특정 기업의 이미지가 포함된 경우, 혹은 알 수 없는 이유로 답변에 실패하기도 합니다. 이런 경우에 재시도하면 해결되기도 하므로 `image_quiz` 함수의 매개변수에 `n_trial=0`과 `max_trial=3`을 추가하고 ❸번 부분에 `try ~ except` 구문으로 처리해 둡니다.
- ❷ 프롬프트를 살펴봅시다. `quiz_prompt`에서 기존의 프롬프트에 토익 리스닝 문제 스타일로 만들어달라는 내용을 추가하고 'Listening:' 뒤에 영어 예시를 추가합니다. 이렇게 요청하면 GPT가 한글과 영어 문제를 모두 출제합니다.
- ❸ 첫 번째 시도가 실패하면 `n_trial`에 1을 더한 값으로 자기 자신 함수(`image_quiz`)를 호출하여 재시도하고, `n_trial`이 3에 도달하면 더 이상 시도하지 않고 예외로 처리합니다.
- ❹ 정상으로 작동하면 결과에서 텍스트만 추출하고 이 함수가 성공했다는 의미로 `True`와 함께 반환합니다.
- ❺ GPT가 생성한 결과가 우리가 지정한 형식과 다를 수도 있습니다. 예를 들어 GPT가 실수로 'Listening:' 이 없는 상태로 답변을 생성했다면 다시 시도하도록 설정합니다. 이 부분을 추출하여 다음 실습에서 TTS로 영어 음성 파일을 MP3로 만들 예정입니다.

2. `image_quiz` 함수를 변경했으므로 호출하는 부분도 수정합니다. 원래 `image_quiz` 함수를 호출할 때 `try ~ except` 구문을 사용했는데 이제 함수 내에서 오류 처리가 이루어지므로 이 구문은 제외합니다. 그 대신 `image_quiz` 함수가 반환하는 두 개의 값 중 두 번째 값을 확인하여 만약 문제 생성에 실패하면 해당 문제는 건너뛰고 다음 문제로 넘어가도록 구성합니다.

```
(... 생략 ...)
txt = ''
no = 1
for g in glob('./data/images/*.jpg'):
    q, is_succeed = image_quiz(g)

    if not is_succeed:
        continue

    divider = f'## 문제 {no}\n\n'
(... 생략 ...)
```

이 코드를 실행한 결과, 다음과 같이 영어로도 문제가 잘 출력되었습니다.

```
## 문제 1

![image](busan_dive.jpg)

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?
- (1) 많은 사람들이 테이블에 앉아 있습니다.
- (2) 배경에 "DIVE 2024 IN BUSAN"이라는 문구가 보입니다.
- (3) 회의실 안에는 식물들이 많이 있습니다.
- (4) 대부분의 사람들이 노트북을 사용하고 있습니다.

Listening: Which of the following descriptions of the image is incorrect?
- (1) Many people are seated at tables.
- (2) The background displays the text "DIVE 2024 IN BUSAN."
- (3) There are many plants inside the conference room.
- (4) Most people are using laptops.

정답: (3) 회의실 안에는 식물들이 보이지 않습니다.

-----

## 문제 2
(... 생략 ...)
```


Do it! 실습 TTS로 영어 듣기 평가 문제 만들기

■ 결과 파일: sec02/image_quiz.py, sec02/tts.ipynb

이제 image_quiz.md 파일에 영어 시험 문제가 생겼습니다. 문제를 읽는 MP3 음성 파일이 있다면 듣기 평가 문제로 활용할 수 있습니다. 오픈AI의 API에는 문장을 음성 파일로 변환하는 TTS(text-to-speech)가 있습니다. 이 기능을 활용해 내가 찍은 사진으로 듣기 평가 문제를 만들어 봅시다.

1. 영어 문제 부분만 추출하기 위해 JSON 파일로 저장하는 기능을 추가합니다. 앞서 생성한 image_quiz.md 파일에서 영어 문제는 'Listening:' 부터 '정답:' 사이에 작성되어 있습니다. 따라서 이 부분만 추출하는 코드를 작성합니다. 이렇게 추출한 영어 문제는 eng 변수에 담고 문제 번호 no와 파일명 filename을 함께 딕셔너리 형태로 저장합니다. 이 딕셔너리를 eng_dict에 하나씩 추가하고 각각 JSON 파일로 저장합니다.

영어 문제만 추출해 JSON 파일로 저장하기

■ image_quiz.py

```
from glob import glob
import json
from openai import OpenAI
(... 생략 ...)

def image_quiz(image_path, n_trial=0, max_trial=3):
    (... 생략 ...)

txt = ''
eng_dict = []
no = 1
for g in glob('./data/images/*.jpg'):
    (... 생략 ...)
    filename = os.path.basename(g)
    (... 생략 ...)
    with open('./chap06/data/images/image_quiz_with_eng.md', 'w', encoding='utf-8') as f:
        f.write(txt)
```

```

# 영어 문제만 추출
eng = q.split('Listening: ')[1].split('정답:')[0].strip()

eng_dict.append({
    'no': no,
    'eng': eng,
    'img': filename
})

# JSON 파일로 저장
with open('./chap06/data/images/image_quiz_eng.json', 'w', encoding='utf-8') as f:
    json.dump(eng_dict, f, ensure_ascii=False, indent=4)

no += 1 # 문제 번호 증가

```

이 코드를 실행하면 기존의 .md 파일 외에 image_quiz_eng.json 파일이 추가로 생성됩니다. 생성된 결과는 다음과 같습니다.

영어 문제만 추출해 JSON 파일로 저장한 결과

image_quiz_eng.json

```

[
  {
    "no": 1,
    "eng": "Which of the following descriptions of the image is incorrect?\n- (1) Many people are seated at tables.\n- (2) The background displays the text \"DIVE 2024 IN BUSAN.\"\n- (3) There are many plants inside the conference room.\n- (4) Most people are using laptops.",
    "img": "busan_dive.jpg"
  },
  (... 생략 ...)
]

```

2. 이제 오픈AI의 TTS 기능을 사용할 준비를 마쳤습니다. TTS를 단계별로 학습하기 위해 주피터 노트북 파일을 생성합니다. 저는 tts.ipynb라는 이름으로 생성했습니다. 우선 오픈AI의 API를 설정합니다.

오픈AI API 설정하기

tts.ipynb (1)

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기
client = OpenAI(api_key=api_key)      # 오픈AI 클라이언트의 인스턴스 생성
```

3. 다음은 오픈AI의 TTS 공식 문서를 활용한 예제입니다. 오픈AI의 TTS는 다양한 목소리를 제공하는데 그중에 alloy를 선택합니다. 음성으로 생성할 내용은 input에 작성합니다. 오픈AI의 TTS에서 반환한 값은 response에 담겨 있는데 이 내용을 .write_to_file() 함수를 사용해 MP3 파일로 저장합니다.

◆ 공식 문서에는 tts-1 모델을 사용하는 예제가 나와 있습니다(<https://platform.openai.com/docs/guides/text-to-speech>). tts-1 모델은 스트리밍 방식으로 오디오를 출력하는 데 최적화되어 있지만 오디오 품질은 tts-1-hd에 비해 떨어집니다. 우리는 MP3 형식으로 출력해야 하므로 이 예제에서는 tts-1-hd를 사용했습니다.

첫 번째 TTS 테스트

tts.ipynb (2)

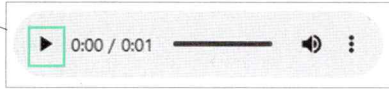
```
# TTS 함수
response = client.audio.speech.create(
    model="tts-1-hd",
    voice="alloy",
    input="Hello world! This is a TTS test.",
)

response.write_to_file("hello_world.mp3")

# 재생
import IPython.display as ipd

ipd.Audio("hello_world.mp3")
```

셀을 실행하면 다음과 같이 hello_world.mp3 파일을 재생할 수 있는 플레이어가 나타납니다. 플레이 버튼을 클릭하니 중성적인 음성이 들립니다.



4. 목소리를 바꿀 수도 있습니다. 목소리를 alloy에서 ash로 바꾸고 자신의 이름을 말하도록 input을 수정합니다. MP3 파일도 기존 파일을 덮어쓰지 않도록 voice 이름을 포함해서 저장합니다.

목소리를 ash로 바꾸고 테스트하기

tts.ipynb (3)

```
# 다른 목소리
voice = "ash"
mp3_file = f"hello_world_{voice}.mp3"

response = client.audio.speech.create(
    model="tts-1-hd",
    voice=voice,
    input=f"Hello world! I'm {voice}. This is a TTS test.",
)

response.write_to_file(mp3_file)

# 재생
import IPython.display as ipd

ipd.Audio(mp3_file)
```

실행해 보면 중저음의 멋진 남성 목소리가 들립니다. 목소리는 ‘alloy’, ‘ash’, ‘coral’, ‘echo’, ‘fable’, ‘onyx’, ‘nova’, ‘sage’, ‘shimmer’ 등이 있습니다. 테스트하면서 마음에 드는 목소리를 선택하면 됩니다.



5. 이제 미리 저장했던 image_quiz_eng.json 파일을 읽습니다.

영어 스크립트 JSON 파일 읽기 (1)

tts.ipynb (4)

```
import json

# JSON 파일 열기
with open('../data/images/image_quiz_eng.json', 'r', encoding='utf-8') as f:
    eng_dict = json.load(f)

eng_dict
```

이 셀을 실행하면 JSON 파일이 딕셔너리 형태로 읽어집니다.

```
[{'no': 1,
  'eng': 'Which of the following descriptions of the image is incorrect?\n- (1) Many people are seated at tables.\n- (2) The background displays the text "DIVE 2024 IN BUSAN."\n- (3) There are many plants inside the conference room.\n- (4) Most people are using laptops.',
  'img': 'busan_dive.jpg'},
 (... 생략 ...)]
```

6. for 문으로 이 딕셔너리를 하나씩 읽어 옵니다. 이 셀을 실행하기 전에 .write_to_file() 내에 지정한 경로가 존재하는지 꼭 확인하세요. 만약 없다면 그 폴더를 만들어 주어야 정상으로 작동합니다.

영어 스크립트 JSON 파일 읽기 (2)

tts.ipynb (5)

```
voices = ['alloy', 'ash', 'coral', 'echo', 'fable', 'onyx', 'nova', 'sage', 'shimmer']

for q in eng_dict:
    no = q['no']
    quiz = q['eng']
    quiz = quiz.replace("- (1)", "- One.\t")
    quiz = quiz.replace("- (2)", "- Two.\t")
    quiz = quiz.replace("- (3)", "- Three.\t")
    quiz = quiz.replace("- (4)", "- Four.\t")
```

```
print(no, quiz)

voice = voices[no % len(voices)]—2

response = client.audio.speech.create(
    model="tts-1-hd",
    voice=voice,
    input=f'#{no}. {quiz}',
)

response.write_to_file(f"../data/audio/{no}.mp3")
```

- ❶ q['eng']에서 영어 문제를 가져와 저장한 후 '-' (1)'을 '- One. \t'로 바꿉니다. 이 부분 없이 음성 파일을 생성했다면 종종 숫자를 생략하고 읽는 문제가 발생합니다. 따라서 번호를 빠뜨리지 않도록 명시적으로 표현합니다.
- ❷ voices로 선택할 수 있는 목소리의 이름을 리스트에 담아 놓고 문제 번호 no가 몇인지에 따라 음성이 선택되도록 합니다. 이렇게 하면 MP3 파일을 각각 다른 목소리로 생성할 수 있습니다.

이제 다음 셀에서 원하는 파일을 선택해 재생해 봅시다.

영어 스크립트 JSON 읽기 (3)

tts.ipynb (6)

```
ipd.Audio(f"../data/audio/1.mp3")
```

내가 찍은 사진으로 그럴싸한 영어 듣기 평가 문제가 생성되었습니다!

